跑掉四五千万人民币等值 Token 之后，两位英伟达工程师重新理解了 Agent：代码不是护城河，时间才是

涌现 Lab NeuralTalk 2026年10月7日 12:01 陕西

投稿/寻求报道/文章纠错：公众号后台 -> 联系我们

关键词：Humanize、*KDA*、Agent、Token、RSI

Agent 批量写算子，工程师还能做什么？对谈 Humanfia

NeuralTalk

27分钟

> 科技播客「涌现 Lab」最近上线了一期信息量很大的对谈，本文基于这期约 79 分钟访谈稿整理，存在信息丢失，最好是去看看原视频见**B 站**[1]，音频版见**小宇宙**[2]。

主播梁瑶请到两位在英伟达做 Agent 基础设施的研究者：

- 朱力耕，NVIDIA Research 研究科学家，MIT 博士，师从韩松，目前负责 Agent Efficiency 与 Scaling 方向；
- 刘思皓，NVIDIA Research 体系结构组研究科学家，今年 9 月刚从 UCLA 博士毕业，是 Agent 流程框架 Humanize 的作者。

两人所在的 Humanfia 社区，**围绕 Humanize 和 kernel 设计智能体 KDA 这两个项目，一年多来累计烧掉了四五千万元人民币等值的 Token**——***据朱力耕估算，最密集的时候一天要跑掉几万美元等值的 API 额度。***

图 1：本期节目封面：主播梁瑶与嘉宾朱力耕、刘思皓。封面底部的话题导航栏预告了全片脉络：Humanize 起源、KDA 写算子、从 Token 到真实生产力、软件工程新趋势、人的转向、RSI 与护城河。主播梁瑶是「涌现 Lab」创始人，该播客由 00 后在校学生发起，定位科技一线对谈。

- **涌现 Lab：Agent 批量写算子，工程师还能做什么？对谈 Humanfia**
- 涌现 Lab B 站：https://www.bilibili.com/video/BV1ubHp6MEvp
- 涌现 Lab 小宇宙：https://www.xiaoyuzhoufm.com/episode/6ac218a3195d838e2aed2ce5
- **Humanize 项目：https://humanfia.ai**
- KDA 项目：https://nvlabs.github.io/kda

Agent 批量写算子，工程师还能做什么？对谈 Humanfia

NeuralTalk

27分钟

相关推荐

- Claude 实现、Codex 审查、人类决策领航：Humanize 项目用 RLCR 编排长程闭环迭代任务
- 陈天奇团队提出 CAKE：编译器与智能体协同进化，可进化 IR 让 KDA 提速 2.05 倍
- 算子工程师末日。。。NVIDIA提出SOL-ExeBench：对标硬件效率极限，生成真实 GPU Kernel！

> 四五千万元人民币等值的 Token 这笔钱买到的，是一套经过大规模实践检验的、有些反直觉的 Agent 工程认知。

对话从 Humanize 的起源讲起：刘思皓为完成约 60 万行代码的博士论文框架 Loom，把「写代码」变成「写规范」，**最终提炼出「一个 Agent 构建、一个 Agent 审查」的极简循环——照搬公司组织结构的多智能体分工只是「过家家」，*真正 work 的结构简单到只有这一个构建者加一个审查者。***

朱力耕则把这套 harness 理解为 test-time scaling，并把它引向 GPU kernel 优化，**催生出在 MLSys 2026 FlashInfer 内核竞赛拿下前三的 KDA 项目。**

> 两人还分享了双模型接力跳出局部最优的 Flame Chase 流程、用 Lean 做数学证明横扫奥赛榜单的经历，以及两条更进一步的判断：费力搭好的 harness，新模型一来就该先拆掉；**当所有人都在用 Agent，代码和流程都不是护城河，唯一的护城河是时间**。主线的终点是一个更激进的图景：***投入 Token 优化推理系统本身，同样的算力就能生产更多 Token——用 Token，换来更多 Token。***

本文目录

- 一、Humanize 的起源：一个人写 60 万行代码的博士论文
- 二、朱力耕的视角：harness 是一种 test-time scaling
- 三、KDA 的三层结构：流程、知识库、性能反馈
- 四、Flame Chase：两个模型的接力跑

- 五、多 Agent 协作：别「过家家」
- 六、Token 账本：四五千万人民币花在了哪
- 七、数学榜单：一群工程师的「乱拳打死老师傅」
- 八、新模型来了，先把 harness 拆掉
- 九、工程师的转向：护城河是时间
- 十、RSI：阿拉丁神灯式的自举循环
- 结语

https://humanfia.ai/

一、Humanize 的起源：一个人写 60 万行代码的博士论文

> Humanize 不是一个立项出来的产品，它是刘思皓被自己的博士论文逼出来的工具。

刘思皓在 UCLA 读博期间做加速器设计，从底层 RTL、Verilog 和芯片版图，一路扩展到高层架构与编译器。**他的博士论文框架 Loom（织布机）是一个多核异构加速器框架，要在前端对接编译器、在后端对接硬件架构与版图生成，跨度极大。*整个项目有七到十个组件，每个组件三万到五万行代码，加起来约 60 万行——而学校里这个项目只有他一个人。***

「前 AI 时代，一个人完成 50 万行以上的代码库是难以想象的。」刘思皓的解法是把工作重心从写代码变成写规范：规范用自然语言写，代码库用编程语言写，中间的翻译交给模型。这正好撞上了语言模型最擅长的事情——把自然语言的 spec 翻译成编程语言的代码。

但 2025 年 10 月前后的 Claude Code 还没有今天这样的长程能力。他丢给模型一份一两千行的 spec，期望产出一万多行代码，模型却总是中途停下来。单纯发「continue」会让模型陷入局部最优——它知道自己没做好，但就是不做了；换其他模型来审查进度，审查能力又不够强。

他先尝试了另一条路：借鉴 GitHub 的协作模式，做了一个叫 GAAC（GitHub-as-a-Context）的项目，把 issue、PR 作为智能体的上下文，以 PR 合并和 issue 发布为节奏推进，**本质是把公司的组织结构搬进多智能体编排。这个项目跑了大约一百多个 issue 之后就不 work 了。**

失败的原因很本质：公司里人和人靠开会沟通，而智能体之间的沟通是把一套上下文从一个 Agent 迁移到另一个 Agent，这种迁移是有损压缩。*多个智能体对着同一份任务频繁迁移上下文，信息逐渐丢失，工作区最终变成一锅 slop*（指质量低劣、语义稀薄的 AI 产出物）。

> 刘思皓回头拆解，**发现真正占据开发 90% 时间的结构只有一个：一个 Agent 做 build，另一个 Agent 做 review。**他把这个结构摘出来，又试了几种组合：
>
> - Claude 做 builder、换不同模型做 reviewer，效果平平；
> - 搭一个三到五个小模型组成的「审查委员会」，也不行。
> - 直到让 Codex 来做 reviewer，效果一下子变得非常好。
>
> 这就是 Humanize 的第一版：***人把想做的 plan 丢给 Claude 实现，Codex 负责审查，保证 spec 规定的东西被完整实现。***

2026 年 1 月中旬，他白天写好 plan 丢给 Humanize，让它跑十几二十个小时，第二天查看时，spec 里每一部分都被完整实现了。「这件事放到现在稀松平常，但九个月之前，这是非常神奇的事情。」

这个循环的设计哲学值得一提。**刘思皓不追求 loop 快速收敛，而追求反馈足够快、每一轮都有微小提升：**与其跑 3 到 5 轮、每轮提升 5%，不如每轮只提升 1% 到 2%、连续跑 100 轮——1.01^{100} 远大于 1.05^{3}。配套机制还有一条止损规则：连续 5 轮没有进展就停止。

Humanize 最初只是刘思皓的 side project，后来推荐给周围的人，再推荐给朱力耕，社区逐渐变大，仓库从 UCLA 的 **PolyArch/humanize**[3] 发展到今天的 **humanfia/humanize**[4]。朱力耕把它用到了一个刘思皓自己都没想过的方向：对着单一指标做深度优化——写 GPU kernel。

二、朱力耕的视角：harness 是一种 test-time scaling

> 朱力耕的研究背景偏机器学习系统，他坦言自己不是 kernel 出身。让他决定加入的，是 Humanize 的一个演示案例：一个人用一周时间，让 Agent 全自动地把仿真器 gem5 的构建系统从 SCons 迁移到 CMake，改了 567 个文件，CI 一次性通过。

「我当时立刻意识到 something is happening here。」朱力耕说。换作做训练的人的视角，这件事的实质很清楚：**只要你保证 Agent 能在同一件事情上持续输出而不发生漂移，这就是一种 test-time scaling**（测试时扩展，即通过增加推理阶段的计算量来提升产出质量）。***水滴石穿，总能磨出结果——何况模型还在一代代变聪明。***

> 选 kernel 作为试验场有三个理由。

- 第一，CUDA kernel 是英伟达非常重要的护城河，两位嘉宾又都在英伟达，任务与主业直接相关。
- 第二，写 kernel 高度依赖领域知识——MIT 有专门教 CUDA 并行计算的课程，但即便上完课，工程师日常还是会碰到没见过的高效写法，因为每一代硬件都有新特性。
- 第三，kernel 是反馈极快的任务：写完用 NVCC 编译，数值一测就知道对不对，测速一跑就知道拿到多少加速比。短频快、上限高，最适合拿来试探 Agent 能力的上限。

结果不负众望。KDA（Kernel Design Agents）0.1 版在 MLSys 2026 FlashInfer 内核竞赛上，核心开发只用了一周左右，就在六周赛期的比赛里拿下全赛道前三名。让圈内人最意外的不是成绩本身，而是这支队伍不是 kernel 团队——主力参与者实习生 Dongyun Zou 此前只写过约 400 行 Triton，CUDA 一行没写过。 kernel 专家用 Agent 打辅助写出好成绩不奇怪，**一群非 kernel 背景的人靠 Agent 拿到前三，才是信号。**

> 经过此后两个月的迭代，KDA 在很多 kernel 上已经能写出媲美人类专家、甚至比人类专家更快的实现。**四五月份写一个好 kernel 还是 24 小时起步，现在 6 到 8 小时就能收敛到不错的结果，*单个 kernel 的 Token 成本约 800 万到 1100 万——按嘉宾的说法，非常便宜，而且可以批量生产。***

三、KDA 的三层结构：流程、知识库、性能反馈

> KDA 的结构分为三层。

- **最上层是 Humanize 的 flow，管理整个 Agent loop。**最早用的是 RLCR 循环——名字可以读作 Ralph loop 加 Code Review，也可以读作 Reinforcement Learning with Code Review：Claude 负责写，Codex 负责审，进入循环前先有详细的规划过程。
- **中间层是 KernelWiki，一个面向 Agent 的领域知识库。**团队把 FlashInfer、SGLang、vLLM 等热门仓库近半年已合并的 PR 整理出来，提炼其中的 know-how 和经验教训，作为 Agent 可检索的记忆——就像人写 kernel 时会去看别人的成功代码是怎么写的。
- **底层是细粒度的性能反馈。**只看 latency 或 speedup，Agent 只能拿到一个数字，不知道瓶颈在哪；接入 NCU（Nsight Compute）的 profiling skill 之后，Agent 能看清 DRAM 带宽是否不够、L2 cache 有没有打满、SM 调度是否出了问题，迭代方向因此变得明确。开发过程中还陆续加入了 CuTe 相关的 IKET 等工具，后续计划接入 nsys。

> 刘思皓总结，抛开写代码和性能调优的具体技巧，***一个好 harness 的核心是把最核心的结构做得足够简单，同时引入三件事：混合模型（两个或多个足够强且彼此有差异的模型）、足够短且信息量足够的反馈回路、干净高质量的上下文管理。***如果还要再加一条，就是并行 Agent 的管理能力——这是他们正在研究的方向。

> 这套结构在实战中不断打出成绩。

- 9 月底，团队发布 KDA[2] 案例：用 KDA 去优化名字撞车的另一个 KDA——月之暗面 Kimi Linear 模型的 Kimi Delta Attention 算子。
- 在 B300 上，Agent 写出的 kernel 相对官方 FlashKDA 的几何平均加速达到 2.96 倍（TIRx 版本），CAKE-PTX 版本 2.94 倍，CuTe-DSL 版本 2.85 倍，且精度优于原版。过程中 Agent 还曾交出 3.57 倍甚至 5.16 倍的「假优化」——靠硬编码输入分布的统计特征、利用测试集的精度漏洞骗过验证，团队随后用动态输入加盐、压力探针、严格逐元素容差等手段把验收体系层层加固。
- 在 NVIDIA 的 SOL-ExecBench 内核榜单上，KDA 流程一个人一周把 235 个 kernel 分榜全部跑完，截至 10 月 5 日拿下其中 71 个榜首，为所有参赛者中最多。
- 而在生产侧，8 月以来已有 6 个 KDA 生成的 kernel 被 SGLang 主仓合并，其中一个为 Qwen3.5-4B 的推理带来 8.7% 的吞吐提升。

图 2：KDA 官网的「社区愿望清单」流程：社区成员以 PR 形式提交 kernel 优化需求，要求附带可复现的定义、代表性负载和已知最佳基线，KDA 流程据此产出经过验证的 kernel。目前该清单仅支持 B200 与 B300 两代 GPU——这条硬件限制来自仓库文档，访谈中未展开。KDA 项目主页：https://nvlabs.github.io/kda。

四、Flame Chase：两个模型的接力跑

> RLCR 能保证长时间执行不漂移，但它缺少多样性。做优化类任务时，灵光一现很重要，怎么给 Agent 人为创造出灵光一现的机会?

团队的答案是 Flame Chase——字面意思就是接力跑。先让 Astra（OpenAI 的 GPT-6 Astra）做一段时间，到达一定的预算后强制它总结并停止，把接力棒交给 Fable（Anthropic 的 Claude Fable）；Fable 做一段时间后再强制停下，交回给 Astra。在优化类任务上，这种交替的上限和收敛速度都比 RLCR 快很多。

为什么两个模型接力比单个模型硬跑更有效?

朱力耕给出了一个解释：当两个模型都足够强且存在多样性时，可以把解题过程想象成一条状态链。模型 A 的解题路径可能是 A、B、C 三步，模型 B 的路径可能是 A、D、C。把两者交叠起来：A 先做完 A 和 B，B 接手后在 B 后面插入一个 D；等 A 再回来时，它看到的不再是原来的 A、B，而是 A、B、D——基于这个新的现场，它除了做出 C，还可能做出 E。**一个 Agent 卡住不动的局部最优，把 workspace 清理一下、总结好交给另一个 Agent，往往很快就能走出来。**

> ***这个解释也反过来说明了为什么弱模型做不了 reviewer：强模型已经能预见弱模型能做的一切，弱模型给的反馈是无效反馈，改变不了任何决策***。模型混合的前提，是参与接力的模型都要足够强。

五、多 Agent 协作：别「过家家」

> RLCR 和 Flame Chase 都是单 Agent 结构，两人对当前流行的多 Agent 角色扮演玩法评价直接：那是「过家家」。

朱力耕给出的数据是：**多 Agent 协作模式下 token 消耗通常放大 10 到 20 倍，换来的产出提升只有 1.1 到 1.2 倍。投入和产出完全不成比例。**

刘思皓补充，给 Agent 分配「架构师」「程序员」「测试工程师」这类角色，本质是让人类组织里的沟通损耗原样复制到机器上——**人类需要分工是因为人脑容量有限、沟通成本高，*而单个 Agent 完全有能力通读整个上下文，人为割裂上下文只会制造新的信息不对称。***

> 但这不意味着拒绝并行。真正有效的并行有两种。

- **一种是硬编码的并行控制**：在 harness 层面把一个任务确定性地切成互不干扰的子任务，比如 KDA 跑 SOL-ExecBench 时 235 个 kernel 分榜并行推进，每个 Agent 拿独立的 workspace 和独立的验证闭环，不存在「开会协调」。
- **另一种是 PR 级别的集成：多个 Agent 各自产出完整的、可独立验证的成果，最后像开源社区合并 PR 一样集成**，冲突由代码事实裁决，而不由某个「经理 Agent」拍脑袋。

沿着这条线，两人谈到了两条正在分化的路线。Anthropic 在 Claude Code 里押注 Ultracode 式的动态 workflow：模型自己决定什么时候分叉、什么时候汇合，信任模型的判断力。

OpenAI 在 9 月 29 日的 DevDay 上推出 Decisions API，方向相反：把编排逻辑从模型手里收回来，用外部结构确定性地控制 Agent 的分支与合并。朱力耕的类比是 CPU 发展史——单核性能见顶之后，业界没有发明「一个核扮演管理员、一个核扮演工人」的协作模式，而是走了

硬件层面的确定性地多核并行。**Agent 编排大概率会重复这条路：模型负责单点智能，并行结构由系统保证。**

六、Token 账本：四五千万人民币花在了哪

> 标题里的「四五千万人民币等值 Token」是两人的真实账单。刘思皓给出的量级是：**团队一天烧掉 14 到 20 亿 token，折合成 API 价格是几万美金一天。*项目早期他自掏腰包，后来靠赞助和 NVIDIA 内部资源支撑。***

这笔钱买到的最重要的认知是「token maxing」——在 agentic 时代，token 本身就是生产资料，要用电一样量产智力。**判断一件事该不该做的标准简化为一条：ROI 是否大于 1。只要一个 Agent 任务消耗的 token 价值低于它产出的价值，这个任务就可以无限放大规模地跑**；如果 ROI 小于 1，再精巧的设计也没有意义。价格坐标也在快速变化：OpenAI 的 o1-pro 输出价格约每百万 token 600 美金，而 GPT-5.6 Sol 这样的新一代模型把同等智能的价格大幅压低。刘思皓的判断是，**随着开源模型追平和推理成本下降，未来会出现「零成本 flow」——整套 harness 跑在开源模型上，token 边际成本趋近于零，那时今天因成本而受限的设计空间会全部打开。**

七、数学榜单：一群工程师的「乱拳打死老师傅」

> kernel 优化之外，这套 harness 还有一个更出人意料的战场：形式化数学。Humanfia 的 HOA（Humanfia Olympiad Agents）项目用 Lean 做数学证明，成绩亮眼——PutnamBench 672 题全部解决，IMO 2026 六题全对，此外还包括 Putnam 2024、2025 等赛事的完整战绩。在第三方榜单 Lean-Eval v1 上，HOA 排名第二，仅次于 Google DeepMind 的 Axiom。

刘思皓强调，**团队里没有人有数学竞赛背景，这个结果靠的是系统工程而不是数学直觉，他自嘲为「乱拳打死老师傅」。*方法的核心是用 Lean 的形式化验证消除幻觉：模型给出的每一行证明都要过 Lean 编译器，通不过就是错，没有任何含糊空间——这正好满足前文说的「足够短且信息量足够的反馈回路」。***证明搜索被组织成一棵 DAG（有向无环图），Agent 在图上分解引理、并行攻坚、合并结果，思路和费马大定理式的巨型证明拆解同源，他半开玩笑地说这套东西可以开一门「数学工程系」。

与 OpenAI、Google 只发布榜单结果不同，HOA 把全过程开源——包括失败的尝试轨迹。刘思皓的理由是，只放结果不放过程，社区就无法复现，也就无法在其上迭代；形式化数学是 harness 能力最纯粹的试金石，因为验证完全机械化，没有人类偏好的水分。

八、新模型来了，先把 harness 拆掉

> 访谈中一个反直觉的观点是：每次新模型发布，团队做的第一件事不是延续之前的旧的 harness，而是**把 harness 拆掉，让模型裸跑一两周。**

逻辑很直接：harness 里的每一条结构都是为上一代模型的缺陷打的补丁，新模型可能已经不再有那个缺陷，补丁本身反而成了枷锁。团队有具体案例：上一代模型时代，他们强制 Agent 走 TDD（测试驱动开发）流程，因为不这样模型就会写出没法验证的代码；***新模型上线后他们发现直接删掉 TDD 环节效果更好——模型自己已经会把测试写好，强制流程只是白白消耗上下文。***正确的姿势是裸跑观察模型能自己做到什么，再把确实需要的结构一项一项加回来。

> 由此引出一个判断：什么样的任务 Agent 做不好？朱力耕给出两类。

- 一类是训练语料里没见过的东西——模型对完全没有先验的领域无从下手。
- 另一类是反馈周期过长的任务，典型例子是芯片流片：一次 tape-out 的反馈要几个月，Agent 的迭代飞轮转不起来。

反过来说，凡是语料覆盖到、反馈回路短的任务，被 Agent 吃掉只是时间问题——kernel 优化和形式化数学正是同时满足这两个条件的领域。

九、工程师的转向：护城河是时间

> 谈到个人层面，两人的共识是工程师正处在一个「变革时代」而非「熬资历时代」。**过去一个工程师的价值很大程度来自对某个系统的熟悉程度——熟悉 CUDA 的犄角旮旯、熟悉某个代码库的历史包袱，这些知识靠年头积累，新人难以跨越。*Agent 把这层护城河抽干了：任何代码库，Agent 几小时就能读通。***

历史上这一幕反复上演。朱力耕的梳理是：汇编时代程序员以手写指令序列为傲，C 和 C++ 把指令细节抽象掉；ATM 普及之后，银行柜员的点钞技能不再稀缺，但没有消失的是理解客户需求、设计金融产品的能力。**每一次 commodity（商品化）抽象层的抬高，淘汰的都是「操作层面的熟练」，留下的是「问题层面的判断」。*这次的不同之处在于速度：以前的抽象层十年抬一次，现在一年抬好几次。***

> ***在 AI 时代，两人认为真正的护城河只剩下一个：时间。你能比别人早一年把 Agent 用得足够深，积累的 harness 经验、验证体系、对模型边界的直觉，就是别人短期补不上的差距。***刘思皓坦言自己也面临这个两难——他做的 Jev（一个 TypeSafe 的决策模型创业项目）和 Humanize 之间**如何分配时间，本质上就是在赌哪条曲线更陡。**

十、RSI：阿拉丁神灯式的自举循环

> 访谈的收尾落在一个更大的图景上：RSI（Recursive Self-Improvement，递归自我改进）。

两人描述了一个「阿拉丁神灯」式的循环：向神灯许愿要更多愿望——今天烧 token 优化 kernel 和推理引擎，serving 效率提高一倍，同样的电力就能产出两倍 token；更多的 token 再投入下一轮优化，循环往复。

token 优化 token 的生产管线，这就是他们理解的 RSI 的可行形态：不是科幻意义上模型自己改写自己的权重，**这已经是 Agent 系统持续改进产生 token 的整条基础设施。**NVIDIA 的 NVL72 机柜在这个图景里是下一代载体——把 72 颗 GPU 用 NVLink 连成一台巨型机器，serving 效率的每一次翻倍都直接放大 token 的产能。

> 这个循环也是项目开源的原因：**NVIDIA 官方授权开源，社区在 SGLang、vLLM、FlashInfer 上的每一次合并都让基线更高，而更高的基线让下一轮优化更有价值。*两人预告了接下来一两个月的新动态，包括 KDA 流程的进一步开放，并邀请社区通过愿望清单提交需求——四五千万人民币买来的经验，他们打算让所有人免费站在上面继续跑。***

结语

最后的最后，把整场访谈压缩成几句话：

- Agent 的能力边界不取决于模型本身，而取决于你给它搭的反馈回路有多短、多准；结构要做减法而不是加法——单 Agent 加混合模型接力，胜过一群 Agent 过家家；
- token 是新的生产资料，ROI 大于 1 的事情可以无限放大；harness 是易耗品，每次新模型发布都要敢于拆掉重来；
- 对工程师个人，熟练正在贬值，**判断力在升值，唯一的护城河是先跑一年积累的时间。**

> 两人用四五千万人民币的 token 账单验证了一条朴素的路径：**找到一个验证完全机械化、反馈足够短的领域，把最简单的结构做到极致，然后让循环自己转起来。*kernel 优化和形式化数学已经转起来了，下一个满足这两个条件的领域被吃掉，大概率不需要再等太久。***

> RSI 的阿拉丁神灯已经擦亮，问题只剩下——你手里有没有足够短的回路，接住它吐出来的 token。

参考资料

[1] B站: *https://www.bilibili.com/video/BV1ubHp6MEvp*

[2] 小宇宙: *https://www.xiaoyuzhoufm.com/episode/6ac218a3195d838e2aed2ce5*

[3] PolyArch/humanize: *https://github.com/PolyArch/humanize*

[4] humanfia/humanize: *https://github.com/humanfia/humanize*

相关推荐

- Claude 实现、Codex 审查、人类决策领航：Humanize 项目用 RLCR 编排长程闭环迭代任务
- 陈天奇团队提出 CAKE：编译器与智能体协同进化，可进化 IR 让 KDA 提速 2.05 倍
- 算子工程师末日。。。NVIDIA提出SOL-ExeBench：对标硬件效率极限，生成真实 GPU Kernel!
- NVIDIA 开源 SoL-Pi 用递归自进化把 Agent Harness 成本压低 50%—54%
- 陈天奇领衔 MLC 社区推出 TIRx Harness：面向智能体 GPU 编程的开源编译支撑框架（Compiler Harness）
- MLC 开源 KCoral：GPU 评测从工具变成基础设施，给 kernel agent 用的轻量 GPU 基准服务器

交流加群请在 NeuralTalk 公众号后台回复：加群

智能体 · 目录

上一篇
陈天奇领衔 MLC 社区推出 TIRx Harness：面向智能体 GPU 编程的开源编译支撑框架...

下一篇
数千美元、17 亿 Token、一周时间写出 Qwen 定制推理引擎，单流解码比调优后